Лабораторная работа №1

Основы литературного программирования

Аристова Арина Олеговна

Содержание

1 Цель работы **5**

2 Задание **6**

3 Теоретическое введение **7**

4 Выполнение лабораторной работы **8**
4.1 Создание проекта DrWatson 8
4.2 Базовая модель экспоненциального роста 9
4.3 Генерация производных форматов 11
4.4 Параметрическая версия модели 13

5 Экспоненциальный рост **17**
5.1 Инициализация проекта и загрузка пакетов 17
5.2 Определение модели 18
5.3 Первый запуск с параметрами по умолчанию 18
5.4 Визуализация результатов 20
5.5 Анализ результатов 21
5.6 Сохранение всех результатов 22

6 Параметрическое исследование экспоненциального роста **23**
6.1 Активация проекта и загрузка пакетов 23
6.2 Определение модели 24
6.3 Определение параметров в Dict 24
6.4 Функция-обертка для запуска одного эксперимента 25
6.5 Запуск базового эксперимента 26
6.6 Визуализация базового эксперимента 27
6.7 Параметрическое сканирование 28
6.8 Запуск всех экспериментов и собор результатов 30
6.9 Анализ и визуализация результатов сканирования 31
6.10 Бенчмаркинг с разными параметрами 35
6.11 Сохранение всех результатов 38

7 Выводы **40**

Список литературы **41**

Список иллюстраций

4.1 Установка пакета DrWatson 8
4.2 Создание проекта с помощью initialize_project 9
4.3 Структура проекта DrWatson 9
4.4 Скрипт scripts/01_exponential_growth.jl 10
4.5 Результаты выполнения скрипта в терминале 10
4.6 График экспоненциального роста 11
4.7 Скрипт tangle.jl 12
4.8 Сгенерированный файл .qmd 12
4.9 Выполнение кода в Jupyter Notebook 13
4.10 Скрипт scripts/02_exponential_growth.jl 14
4.11 Результаты параметрического сканирования в терминале 15
4.12 Сравнительные графики для разных значений α 15
4.13 Jupyter Notebook параметрической версии 16

Список таблиц

1 Цель работы

Освоение методологии литературного программирования: создание самодокументируемого кода, его компиляция в исполняемые файлы (чистый код, Jupyter Notebook) и генерация технической документации (Quarto) с возможностью параметризации вычислений.

2 Задание

- Создать рабочий каталог для всего курса.
- Создать рабочее пространство для программ в рамках лабораторной работы.
- Выполнить все задания по тексту лабораторной работы.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
  - чистый код;
  - jupyter notebook;
  - документацию в формате Quarto.
- Выполнить код из jupyter notebook.
- Интегрировать документацию в формате Quarto в отчёт.
- Добавить в код в литературном стиле вычисление для набора параметров.
- Сгенерировать из литературного кода с параметрами:
  - чистый код;
  - jupyter notebook;
  - документацию в формате Quarto.
- Выполнить код из jupyter notebook с параметрами.
- Интегрировать документацию с параметрами в формате Quarto в отчёт.

3 Теоретическое введение

Julia — высокоуровневый свободный язык программирования с динамической типизацией, созданный для математических вычислений. Эффективен также и для написания программ общего назначения. Синтаксис языка схож с синтаксисом других математических языков, однако имеет некоторые существенные отличия.

Экспоненциальный рост — это процесс увеличения величины, при котором скорость роста в каждый момент времени пропорциональна текущему значению этой величины. Чем больше система, тем быстрее она растет.

Дифференциальное уравнение модели:

$$\frac{du}{dt} = \alpha u$$

Решение уравнения:

$$u(t) = u_0 e^{\alpha t}$$

Экспоненциальный рост — идеализированная модель. В реальности он не может продолжаться бесконечно из-за ограниченности ресурсов. После некоторого времени рост обычно замедляется и переходит в логистический рост (S-образная кривая).

4 Выполнение лабораторной работы

4.1 Создание проекта DrWatson

Для выполнения работы необходимо создать проект с помощью пакета DrWatson (рис. 4.1, рис. 4.2).

```
julia> using Pkg

julia> Pkg.add("DrWatson")
┌ Warning: attempting to remove probably stale pidfile
│   path = "C:\\Users\\arist\\.julia\\registries\\.pid"
└ @ FileWatching.Pidfile C:\Users\arist\AppData\Local\Programs\J
ib\v1.10\FileWatching\src\pidfile.jl:273
    Updating registry at `C:\Users\arist\.julia\registries\Gener
   Resolving package versions...
   Installed ChunkCodecLibZstd ─── v1.0.0
   Installed ChunkCodecCore ────── v1.0.1
   Installed HashArrayMappedTries - v0.2.0
   Installed ChunkCodecLibZlib ─── v1.0.0
   Installed JLD2 ──────────────── v0.6.3
   Installed ScopedValues ──────── v1.5.0
   Installed DrWatson ──────────── v2.19.1
   Installed FileIO ────────────── v1.18.0
    Updating `C:\Users\arist\.julia\environments\v1.10\Project.t
  [634d3b9d] + DrWatson v2.19.1
    Updating `C:\Users\arist\.julia\environments\v1.10\Manifest.
  [0b6fb165] + ChunkCodecCore v1.0.1
  [4c0bbee4] + ChunkCodecLibZlib v1.0.0
  [55437552] + ChunkCodecLibZstd v1.0.0
  [634d3b9d] + DrWatson v2.19.1
  [5789e2e9] + FileIO v1.18.0
  [076d061b] + HashArrayMappedTries v0.2.0
```

Рисунок 4.1: Установка пакета DrWatson

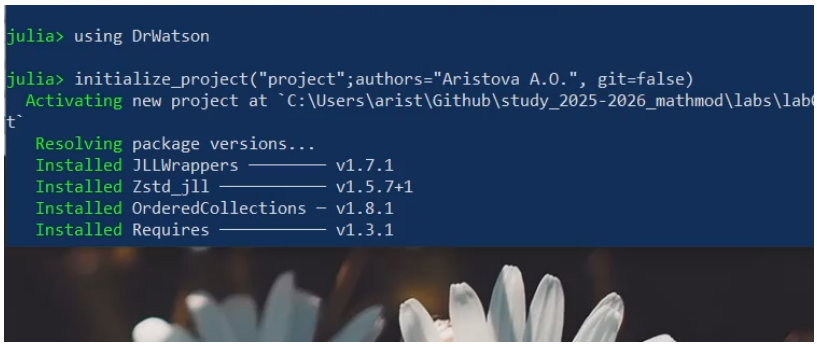


Рисунок 4.2: Создание проекта с помощью initialize_project

Итоговая структура проекта DrWatson (рис. 4.3).

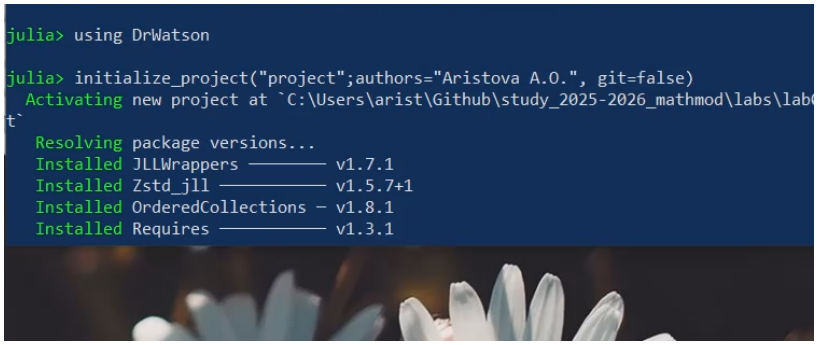


Рисунок 4.3: Структура проекта DrWatson

4.2 Базовая модель экспоненциального роста

Создан скрипт `scripts/01_exponential_growth.jl` в литературном стиле (рис. 4.4).

```julia
# # Экспоненциальный рост
# **Цель:** Исследовать решение уравнения du/dt = αu.
#
# ## Инициализация проекта и загрузка пакетов
using DrWatson
quickactivate(joinpath(@__DIR__, "..", "project"))

using DifferentialEquations
using Plots
using DataFrames
using JLD2

# script_name = splitext(basename(PROGRAM_FILE))[1]
script_name = "01_exponential_growth"
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))
# ## Определение модели
# Уравнение экспоненциального роста:
#
# du
# dt = αu, u(0) = u0
function exponential_growth!(du, u, p, t)
    α = p
    du[1] = α * u[1]
end
# ## Первый запуск с параметрами по умолчанию
# Зададим начальные параметры:
u0 = [1.0] # начальная популяция
α = 0.3 # скорость роста
tspan = (0.0, 10.0) # временной интервал
prob = ODEProblem(exponential_growth!, u0, tspan, α)
sol = solve(prob, Tsit5(), saveat=0.1)
```

Рисунок 4.4: Скрипт scripts/01_exponential_growth.jl

Скрипт выполнен, получены результаты: таблица значений и аналитическое время удвоения (рис. 4.5).

```
PS C:\Users\arist\Github\study_2025-2026_mathmod\labs\lab01\project> julia --project=. scripts/01_exponential_growth.jl
Первые 5 строк результатов:
5×2 DataFrame
 Row │ t        u
     │ Float64  Float64
─────┼──────────────────
   1 │     0.0  1.0
   2 │     0.1  1.03045
   3 │     0.2  1.06184
   4 │     0.3  1.09417
   5 │     0.4  1.1275

Аналитическое время удвоения: 2.31
PS C:\Users\arist\Github\study_2025-2026_mathmod\labs\lab01\project>
```

Рисунок 4.5: Результаты выполнения скрипта в терминале

Получен график экспоненциального роста (рис. 4.6).

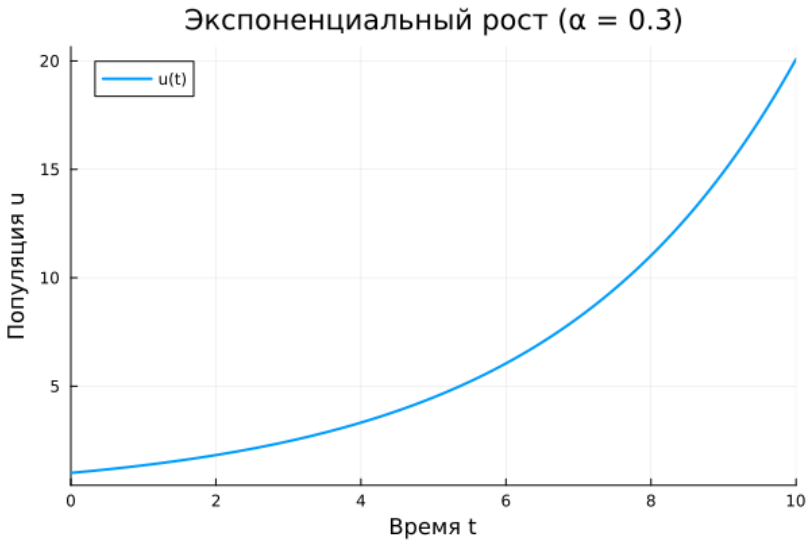


Рисунок 4.6: График экспоненциального роста

4.3 Генерация производных форматов

Создан скрипт `scripts/tangle.jl` для генерации производных форматов (рис. 4.7).

```julia
#!/usr/bin/env julia
# tangle.jl - Генератор отчетов из Literate-скриптов
# Использование: julia tangle.jl <путь_к_скрипту>
using DrWatson
@quickactivate # Активирует текущий проект DrWatson
using Literate
function main()
    if length(ARGS) == 0
        println("""
        Использование: julia tangle.jl <путь_к_скрипту>
        Примеры:
        julia tangle.jl scripts/lab1.jl
        """)
        return
    end


    script_path = ARGS[1]
    if !isfile(script_path)
        error("Файл не найден: $script_path")
    end

    # Пути и имена
    script_dir = dirname(script_path)
    script_name = splitext(basename(script_path))[1]
    println("Генерация из: $script_path")

    # Чистый скрипт (без комментариев)
    scripts_dir = scriptsdir(script_name)
    Literate.script(script_path, scripts_dir; credit=false)
    println(" ✓ Чистый скрипт: $(scripts_dir)/$(script_name).jl")

    # Quarto-документ
    quarto_dir = projectdir("markdown", script_name);
    Literate.markdown(script_path, quarto_dir; flavor=Literate.QuartoFlavor(), name=script_name, credit=
    println(" ✓ Quarto: $(quarto_dir)/$(script_name).qmd")
```

Рисунок 4.7: Скрипт tangle.jl

В результате получены файлы в форматах `.qmd` и `.ipynb` (рис. 4.8, рис. 4.9).

```markdown
## Инициализация проекта и загрузка пакетов

```{julia}
using DrWatson
@quickactivate "project"
using DifferentialEquations
using Plots
using DataFrames
using JLD2

script_name = "01_exponential_growth"
mkpath(plotsdir(script_name))   # создать папку plots/01_exponential_growth/
mkpath(datadir(script_name))    # создать папку data/01_exponential_growth/
```

Определение модели
Уравнение экспоненциального роста: du/dt = alpha*u, u(0) = u0
```

Рисунок 4.8: Сгенерированный файл .qmd

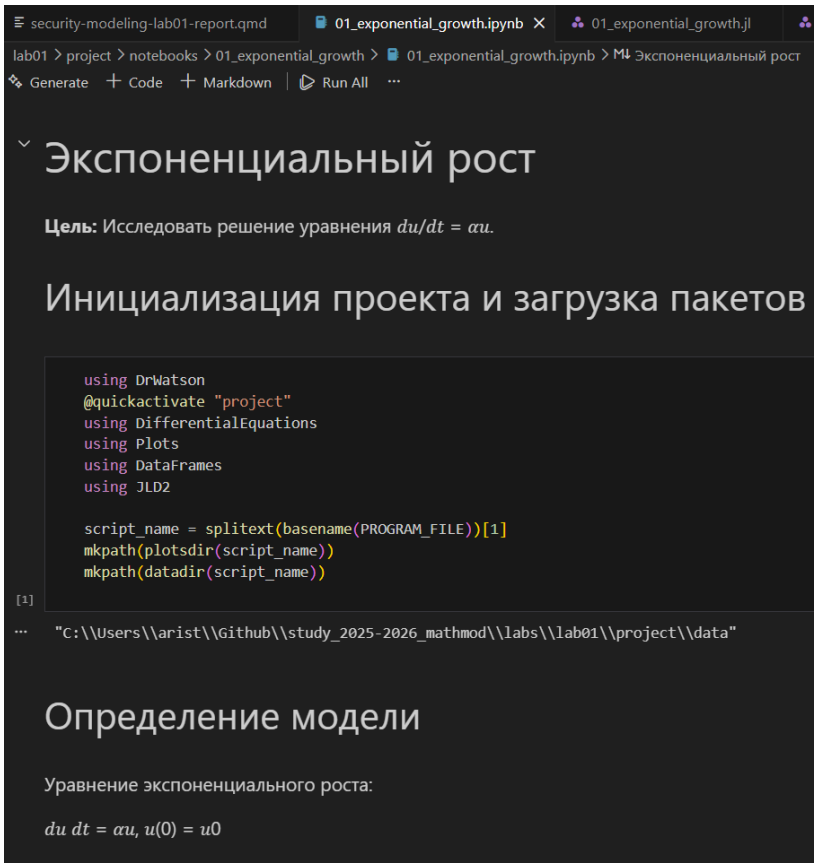
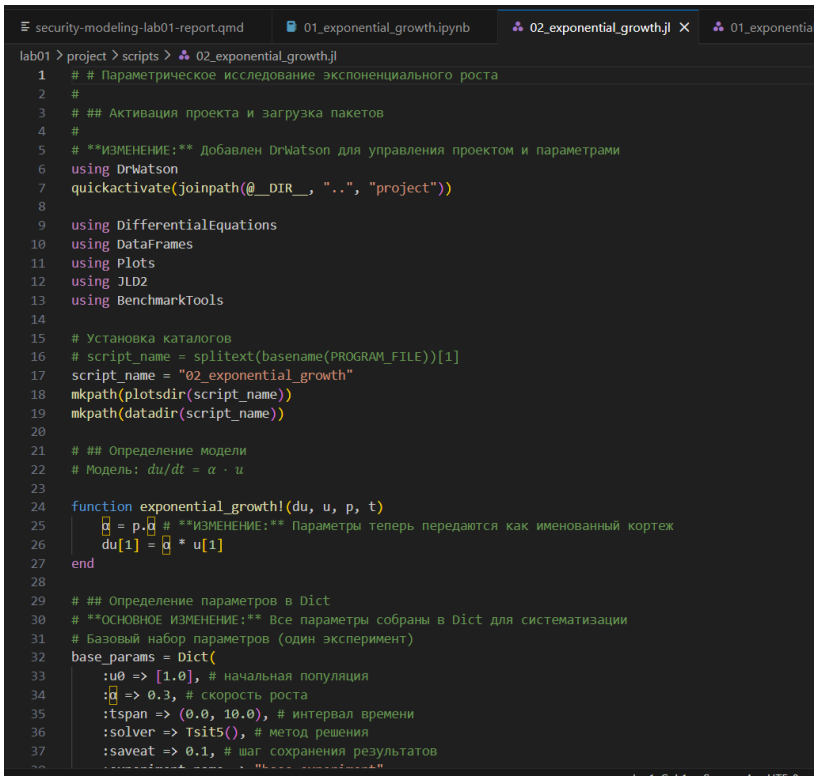


Рисунок 4.9: Выполнение кода в Jupyter Notebook

4.4 Параметрическая версия модели

Создан скрипт `scripts/02_exponential_growth.jl`, реализующий параметрическое исследование модели для набора значений параметра α (рис. 4.10).



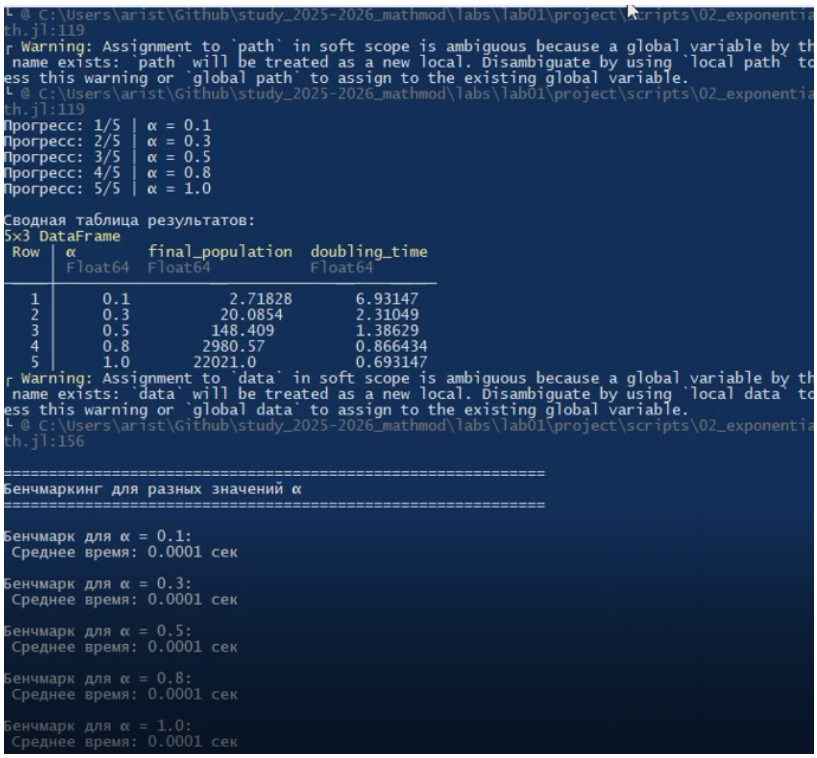
security-modeling-lab01-report.qmd | 01_exponential_growth.ipynb | 02_exponential_growth.jl | 01_exponentia

lab01 > project > scripts > 02_exponential_growth.jl

```julia
# # Параметрическое исследование экспоненциального роста
#
# ## Активация проекта и загрузка пакетов
#
# **ИЗМЕНЕНИЕ:** Добавлен DrWatson для управления проектом и параметрами
using DrWatson
quickactivate(joinpath(@__DIR__, "..", "project"))

using DifferentialEquations
using DataFrames
using Plots
using JLD2
using BenchmarkTools

# Установка каталогов
# script_name = splitext(basename(PROGRAM_FILE))[1]
script_name = "02_exponential_growth"
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))

# ## Определение модели
# Модель: du/dt = α · u

function exponential_growth!(du, u, p, t)
    α = p.α # **ИЗМЕНЕНИЕ:** Параметры теперь передаются как именованный кортеж
    du[1] = α * u[1]
end

# ## Определение параметров в Dict
# **ОСНОВНОЕ ИЗМЕНЕНИЕ:** Все параметры собраны в Dict для систематизации
# Базовый набор параметров (один эксперимент)
base_params = Dict(
    :u0 => [1.0], # начальная популяция
    :α => 0.3, # скорость роста
    :tspan => (0.0, 10.0), # интервал времени
    :solver => Tsit5(), # метод решения
    :saveat => 0.1, # шаг сохранения результатов
```

Рисунок 4.10: Скрипт scripts/02_exponential_growth.jl

Скрипт выполнен, получена сводная таблица результатов для всех значений α (рис. 4.11).



```
└ @ C:\Users\arist\Github\study_2025-2026_mathmod\labs\lab01\project\scripts\02_exponentia
th.jl:119
┌ Warning: Assignment to `path` in soft scope is ambiguous because a global variable by th
 name exists: `path` will be treated as a new local. Disambiguate by using `local path` to
ess this warning or `global path` to assign to the existing global variable.
└ @ C:\Users\arist\Github\study_2025-2026_mathmod\labs\lab01\project\scripts\02_exponentia
th.jl:119
Прогресс: 1/5 | α = 0.1
Прогресс: 2/5 | α = 0.3
Прогресс: 3/5 | α = 0.5
Прогресс: 4/5 | α = 0.8
Прогресс: 5/5 | α = 1.0

Сводная таблица результатов:
5×3 DataFrame
 Row │ α        final_population  doubling_time
     │ Float64  Float64           Float64
─────┼──────────────────────────────────────────
   1 │     0.1           2.71828       6.93147
   2 │     0.3           20.0854       2.31049
   3 │     0.5           148.409       1.38629
   4 │     0.8           2980.57       0.866434
   5 │     1.0           22021.0       0.693147
┌ Warning: Assignment to `data` in soft scope is ambiguous because a global variable by th
 name exists: `data` will be treated as a new local. Disambiguate by using `local data` to
ess this warning or `global data` to assign to the existing global variable.
└ @ C:\Users\arist\Github\study_2025-2026_mathmod\labs\lab01\project\scripts\02_exponentia
th.jl:156

============================================================
Бенчмаркинг для разных значений α
============================================================

Бенчмарк для α = 0.1:
 Среднее время: 0.0001 сек

Бенчмарк для α = 0.3:
 Среднее время: 0.0001 сек

Бенчмарк для α = 0.5:
 Среднее время: 0.0001 сек

Бенчмарк для α = 0.8:
 Среднее время: 0.0001 сек

Бенчмарк для α = 1.0:
 Среднее время: 0.0001 сек
```

Рисунок 4.11: Результаты параметрического сканирования в терминале

Получены сравнительные графики для различных значений α (рис. 4.12).

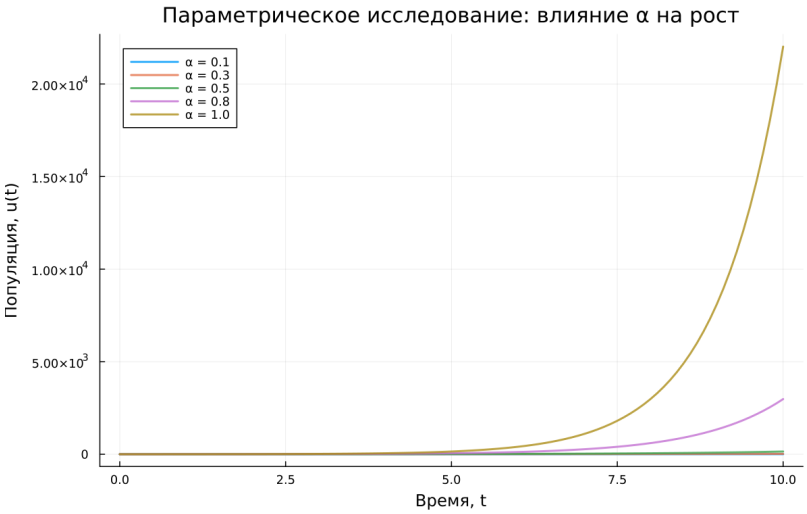


Рисунок 4.12: Сравнительные графики для разных значений α

Параметрическая версия выполнена в Jupyter Notebook (рис. 4.13).

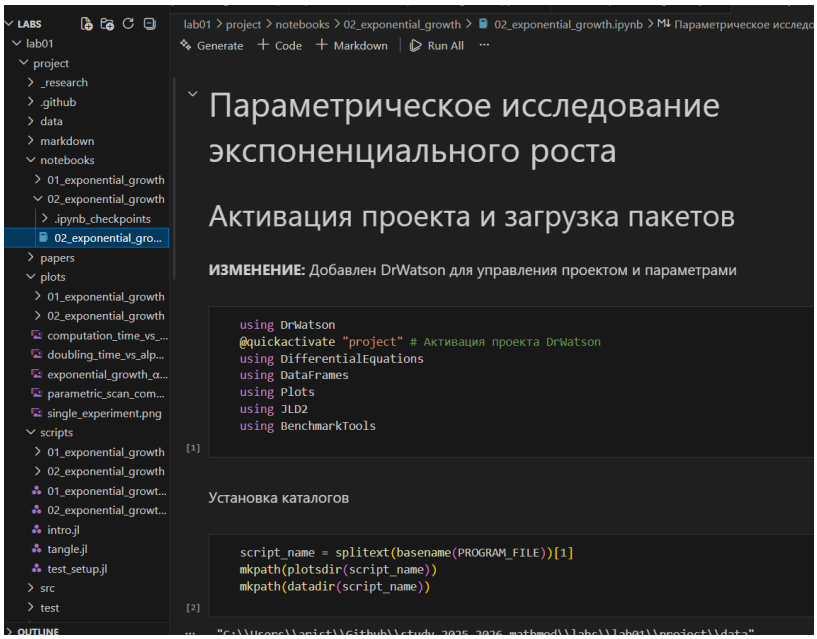


Рисунок 4.13: Jupyter Notebook параметрической версии

5 Экспоненциальный рост

Цель: Исследовать решение уравнения □□/□□ = □□.

5.1 Инициализация проекта и загрузка пакетов

```
using Pkg
Pkg.activate(joinpath(@__DIR__, "..", "project"))
Pkg.instantiate()

using DrWatson
using DifferentialEquations
using Plots
using DataFrames
using JLD2

script_name = splitext(basename(PROGRAM_FILE))[1]
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))
```

```
  Activating project at `C:\Users\arist\Github\study_2025-2026_mathmod\labs\lab01\pro

"C:\\Users\\arist\\Github\\study_2025-2026_mathmod\\labs\\lab01\\project\\data"
```

5.2 Определение модели

Уравнение экспоненциального роста:

$\frac{du}{dt} = \alpha u, u(0) = u_0$

```
function exponential_growth!(du, u, p, t)
    α = p
    du[1] = α * u[1]
end
```

```
exponential_growth! (generic function with 1 method)
```

5.3 Первый запуск с параметрами по умолчанию

Зададим начальные параметры:

```
u0 = [1.0] # начальная популяция
α = 0.3 # скорость роста
tspan = (0.0, 10.0) # временной интервал
prob = ODEProblem(exponential_growth!, u0, tspan, α)
sol = solve(prob, Tsit5(), saveat=0.1)
```

```
retcode: Success
Interpolation: 1st order linear
t: 101-element Vector{Float64}:
  0.0
  0.1
  0.2
  0.3
  0.4
```

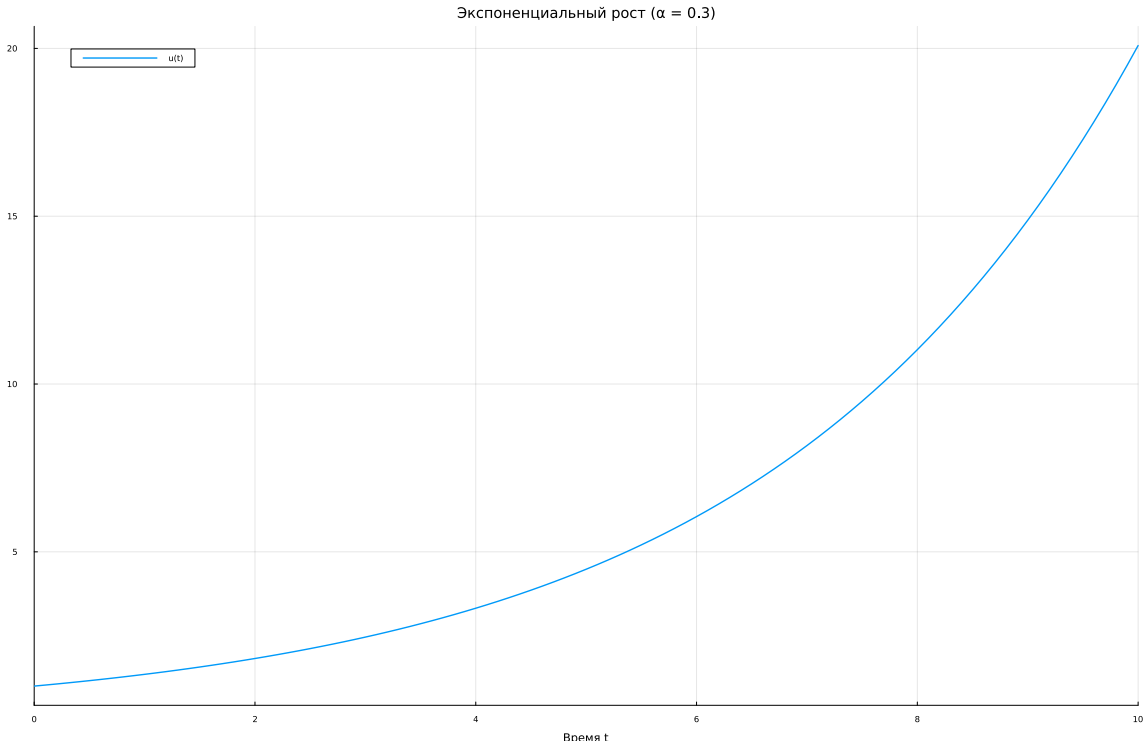
```
  0.5
  0.6
  0.7
  0.8
  0.9
  1.0
  1.1
  1.2
  ⋮
  8.9
  9.0
  9.1
  9.2
  9.3
  9.4
  9.5
  9.6
  9.7
  9.8
  9.9
 10.0
u: 101-element Vector{Vector{Float64}}:
 [1.0]
 [1.030454533950446]
 [1.0618365551529674]
 [1.09417430287941]
 [1.1274968605386673]
 [1.161834232745065]
 [1.197217347699063]
```

```
[1.2336780571872568]
[1.2712491879500347]
[1.3099646073864084]
[1.349859018827382]
[1.3909682733549678]
[1.433329396499376]
⋮
[14.439892233074882]
[14.879627505035817]
[15.332755966505253]
[15.799688704153079]
[16.280848966976613]
[16.776672166300525]
[17.287605875776944]
[17.814109831385306]
[18.356655931432517]
[18.91572823655281]
[19.491822969707833]
[20.085448516186737]
```

5.4 Визуализация результатов

Построим график решения:

```
plot(sol, label="u(t)", xlabel="Время t", ylabel="Популяция u", title="Экспоненциальн
```



Сохраним график в папку plots

```
savefig(plotsdir(script_name, "exponential_growth_□=$□.png"))
```

```
"C:\\Users\\arist\\Github\\study_2025-2026_mathmod\\labs\\lab01\\project\\plots\\expo
```

5.5 Анализ результатов

Создадим таблицу с данными:

```
df = DataFrame(t=sol.t, u=first.(sol.u))
println("Первые 5 строк результатов:")
println(first(df, 5))
```

```
Первые 5 строк результатов:
5×2 DataFrame
 Row │ t        u
```

```
     │ Float64  Float64
─────┼──────────────────
   1 │     0.0  1.0
   2 │     0.1  1.03045
   3 │     0.2  1.06184
   4 │     0.3  1.09417
   5 │     0.4  1.1275
```

Вычислим удвоение популяции:

```
u_final = last(sol.u)[1]
doubling_time = log(2) / □
println("\nАналитическое время удвоения: ", round(doubling_time; digits=2))
```

```
Аналитическое время удвоения: 2.31
```

5.6 Сохранение всех результатов

```
@save datadir(script_name, "all_results.jld2") df
```

6 Параметрическое исследование экспоненциального роста

6.1 Активация проекта и загрузка пакетов

ИЗМЕНЕНИЕ: Добавлен DrWatson для управления проектом и параметрами

```
using Pkg
Pkg.activate(joinpath(@__DIR__, "..", "project"))
Pkg.instantiate()

using DrWatson
using DifferentialEquations
using Plots
using DataFrames
using JLD2
using BenchmarkTools
```

```
Activating project at `C:\Users\arist\Github\study_2025-2026_mathmod\labs\lab01\pro
```

Установка каталогов

```
script_name = splitext(basename(PROGRAM_FILE))[1]
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))
```

```
"C:\\Users\\arist\\Github\\study_2025-2026_mathmod\\labs\\lab01\\project\\data"
```

6.2 Определение модели

Модель: □□/□□ = □ □ □

```
function exponential_growth!(du, u, p, t)
    □ = p.□ # **ИЗМЕНЕНИЕ:** Параметры теперь передаются как именованный кортеж
    du[1] = □ * u[1]
end
```

```
exponential_growth! (generic function with 1 method)
```

6.3 Определение параметров в Dict

ОСНОВНОЕ ИЗМЕНЕНИЕ: Все параметры собраны в Dict для систематизации Базовый набор параметров (один эксперимент)

```
base_params = Dict(
    :u0 => [1.0], # начальная популяция
    :□ => 0.3, # скорость роста
    :tspan => (0.0, 10.0), # интервал времени
    :solver => Tsit5(), # метод решения
    :saveat => 0.1, # шаг сохранения результатов
    :experiment_name => "base_experiment"
```

```
)

println("Базовые параметры эксперимента:")
for (key, value) in base_params
    println(" $key = $value")
end
```

```
Базовые параметры эксперимента:
 □ = 0.3
 u0 = [1.0]
 saveat = 0.1
 solver = Tsit5{typeof(OrdinaryDiffEqCore.trivial_limiter!), typeof(OrdinaryDiffEqCor
 experiment_name = base_experiment
 tspan = (0.0, 10.0)
```

6.4 Функция-обертка для запуска одного эксперимента

ИСПРАВЛЕНИЕ: Возвращаем Dict со строковыми ключами

```
function run_single_experiment(params::Dict)
    @unpack u0, □, tspan, solver, saveat = params
    prob = ODEProblem(exponential_growth!, u0, tspan, (□=□,)) # Создаем и решаем зада
    sol = solve(prob, solver; saveat=saveat)
    final_population = last(sol.u)[1] # Анализ результатов
    doubling_time = log(2) / □
    return Dict(
        "solution" => sol,
```

```
        "time_points" => sol.t,
        "population_values" => first.(sol.u),
        "final_population" => final_population,
        "doubling_time" => doubling_time,
        "parameters" => params # Сохраняем исходные параметры
    ) # Используем строки как ключи для совместимости с DrWatson
end
```

```
run_single_experiment (generic function with 1 method)
```

6.5 Запуск базового эксперимента

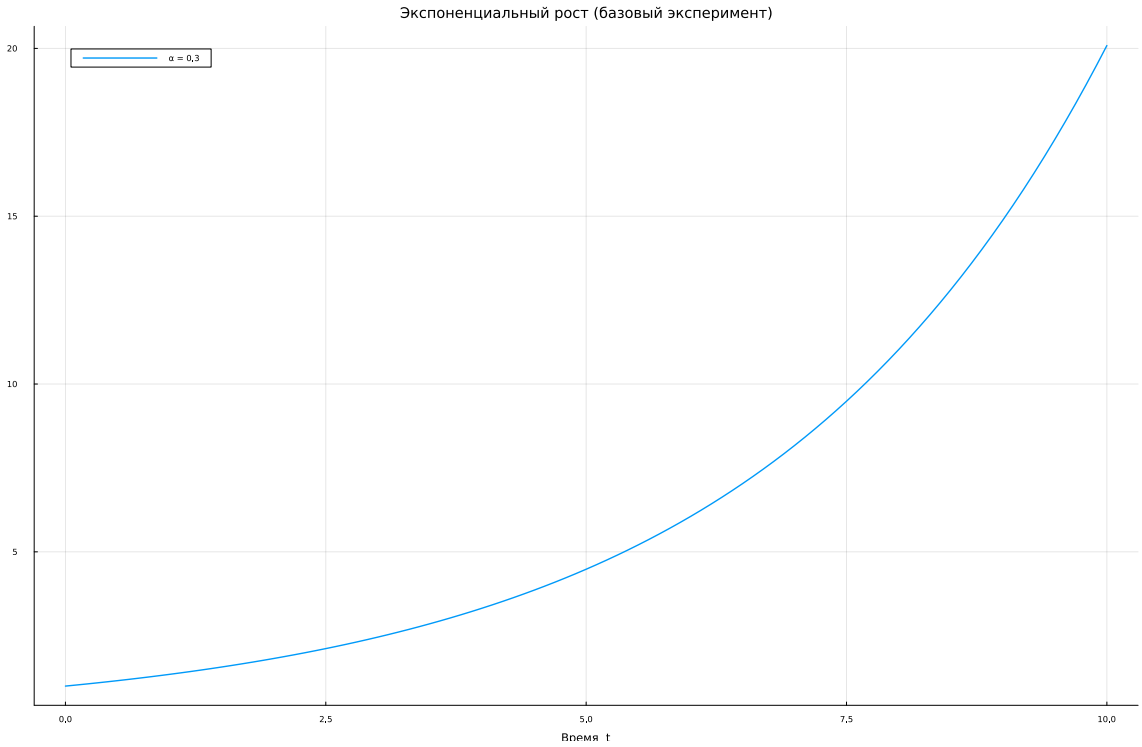
ИЗМЕНЕНИЕ: Используем produce_or_load для автоматического кэширования

```
data, path = produce_or_load(
    datadir(script_name, "single"), # Папка для сохранения
    base_params, # Параметры эксперимента
    run_single_experiment, # Функция для выполнения
    prefix = "exp_growth", # Префикс имени файла
    tag = false, # Не добавлять git-тег
    verbose = true
)
println("\nРезультаты базового эксперимента:")

println(" Финальная популяция: ", data["final_population"])
println(" Время удвоения: ", round(data["doubling_time"]; digits=2))
println(" Файл результатов: ", path)
```

```
Результаты базового эксперимента:
 Финальная популяция: 20.085448516186737
 Время удвоения: 2.31
 Файл результатов: C:\Users\arist\Github\study_2025-2026_mathmod\labs\lab01\project\d
```

6.6 Визуализация базового эксперимента

```julia
p1 = plot(data["time_points"], data["population_values"],
    label="□ = $(base_params[:□])",
    xlabel="Время, t",
    ylabel="Популяция, u(t)",
    title="Экспоненциальный рост (базовый эксперимент)",
    lw=2,
    legend=:topleft,
    grid=true
)
```



Сохраним график в папку plots

```
savefig(plotsdir(script_name, "single_experiment.png"))
```

```
"C:\\Users\\arist\\Github\\study_2025-2026_mathmod\\labs\\lab01\\project\\plots\\sing
```

6.7 Параметрическое сканирование

НОВАЯ СЕКЦИЯ: Исследование влияния параметра α Сетка параметров для сканирования

```
param_grid = Dict(
    :u0 => [[1.0]], # фиксируем начальное условие
    :□ => [0.1, 0.3, 0.5, 0.8, 1.0], # исследуемые значения скорости роста
    :tspan => [(0.0, 10.0)], # фиксируем интервал времени
    :solver => [Tsit5()], # фиксируем метод решения
```

```
    :saveat => [0.1], # фиксируем шаг сохранения
    :experiment_name => ["parametric_scan"]
)
```

```
Dict{Symbol, Vector} with 6 entries:
  :□               => [0.1, 0.3, 0.5, 0.8, 1.0]
  :u0              => [[1.0]]
  :saveat          => [0.1]
  :solver          => [Tsit5{typeof(trivial_limiter!), typeof(trivial_limiter!)…
  :experiment_name => ["parametric_scan"]
  :tspan           => [(0.0, 10.0)]
```

Генерация всех комбинаций параметров

```
all_params = dict_list(param_grid)

println("\n" * "="^60)
println("ПАРАМЕТРИЧЕСКОЕ СКАНИРОВАНИЕ")
println("Всего комбинаций параметров: ", length(all_params))
println("Исследуемые значения □: ", param_grid[:□])
println("="^60)
```

```
============================================================
ПАРАМЕТРИЧЕСКОЕ СКАНИРОВАНИЕ
Всего комбинаций параметров: 5
Исследуемые значения □: [0.1, 0.3, 0.5, 0.8, 1.0]
============================================================
```

6.8 Запуск всех экспериментов и собор результатов

НОВАЯ СЕКЦИЯ: Автоматический запуск и сохранение всех вариантов

```
all_results = []
all_dfs = []
for (i, params) in enumerate(all_params)
    println("Прогресс: $i/$(length(all_params)) | □ = $(params[:□])")
    data, path = produce_or_load(
    datadir(script_name, "parametric_scan"), # Данные
        params, # Текущий набор параметров
        run_single_experiment, # Функция для выполнения
        prefix = "scan", # Префикс имени файла
        tag = false,
        verbose = false # Не выводить подробности для каждого запуска
    ) # Автоматическое сохранение/загрузка каждого эксперимента

    result_summary = merge(
        params,
        Dict(
            :final_population => data["final_population"],
            :doubling_time => data["doubling_time"],
            :filepath => path # Путь к сохраненным данным
        )
    ) # Сохраняем сводные результаты (используем символы для параметров, но данные из
    push!(all_results, result_summary)

    df = DataFrame(
        t = data["time_points"],
```

```
        u = data["population_values"],
        □ = fill(params[:□], length(data["time_points"]))
    ) # Сохраняем полные данные для визуализации
    push!(all_dfs, df)
end
```

```
Прогресс: 1/5 | □ = 0.1
Прогресс: 2/5 | □ = 0.3
Прогресс: 3/5 | □ = 0.5
Прогресс: 4/5 | □ = 0.8
Прогресс: 5/5 | □ = 1.0
```

6.9 Анализ и визуализация результатов сканирования

НОВАЯ СЕКЦИЯ: Сравнительный анализ всех экспериментов Сводная таблица результатов

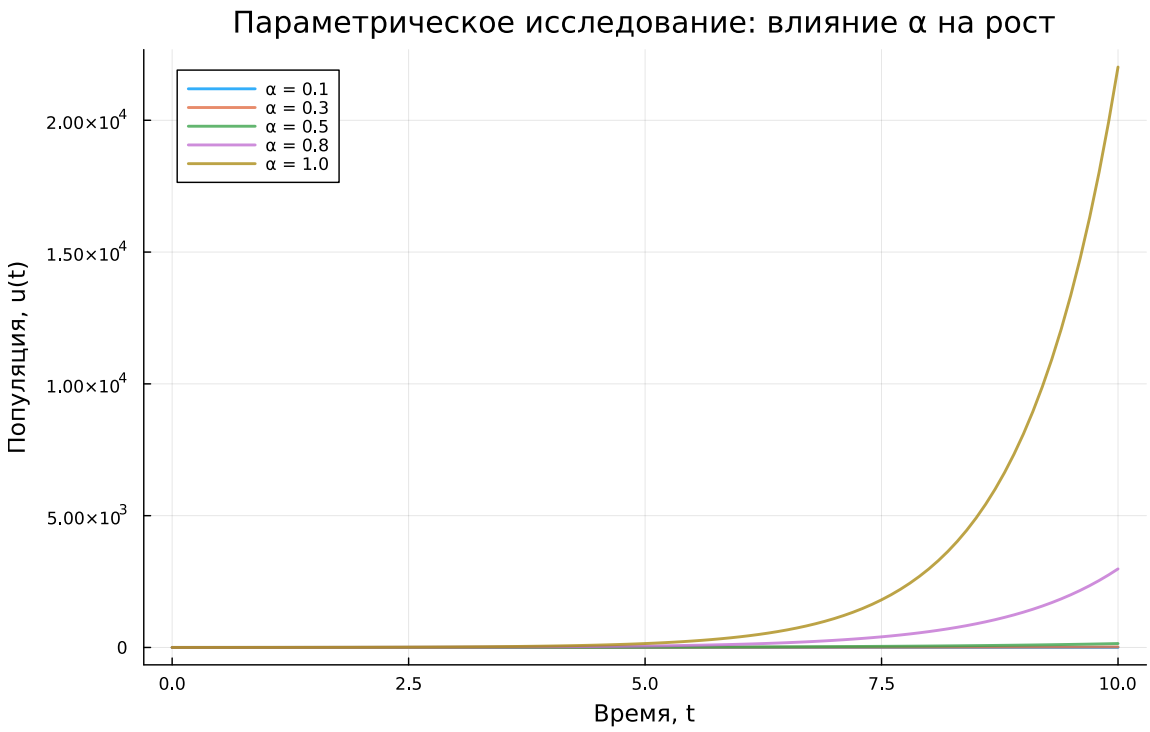
```
results_df = DataFrame(all_results)
println("\nСводная таблица результатов:")
println(results_df[!, [:□, :final_population, :doubling_time]])
```

```
Сводная таблица результатов:
5×3 DataFrame
 Row │ □        final_population  doubling_time
     │ Float64  Float64           Float64
─────┼──────────────────────────────────────────
   1 │     0.1           2.71828       6.93147
   2 │     0.3          20.0854        2.31049
```

3	0.5	148.409	1.38629
4	0.8	2980.57	0.866434
5	1.0	22021.0	0.693147

Сравнительный график всех траекторий

```
p2 = plot(size=(800, 500), dpi=150)
for params in all_params
    data, _ = produce_or_load(
        datadir(script_name, "parametric_scan"),
        params,
        run_single_experiment,
        prefix = "scan"
    ) # Загружаем данные (они уже есть на диске)
    plot!(p2, data["time_points"], data["population_values"],
        label="□ = $(params[:□])",
        lw=2,
        alpha=0.8
    )
end




plot!(p2,
    xlabel="Время, t",
    ylabel="Популяция, u(t)",
    title="Параметрическое исследование: влияние □ на рост",
    legend=:topleft,
    grid=true
)
```



Сохраним график в папку plots

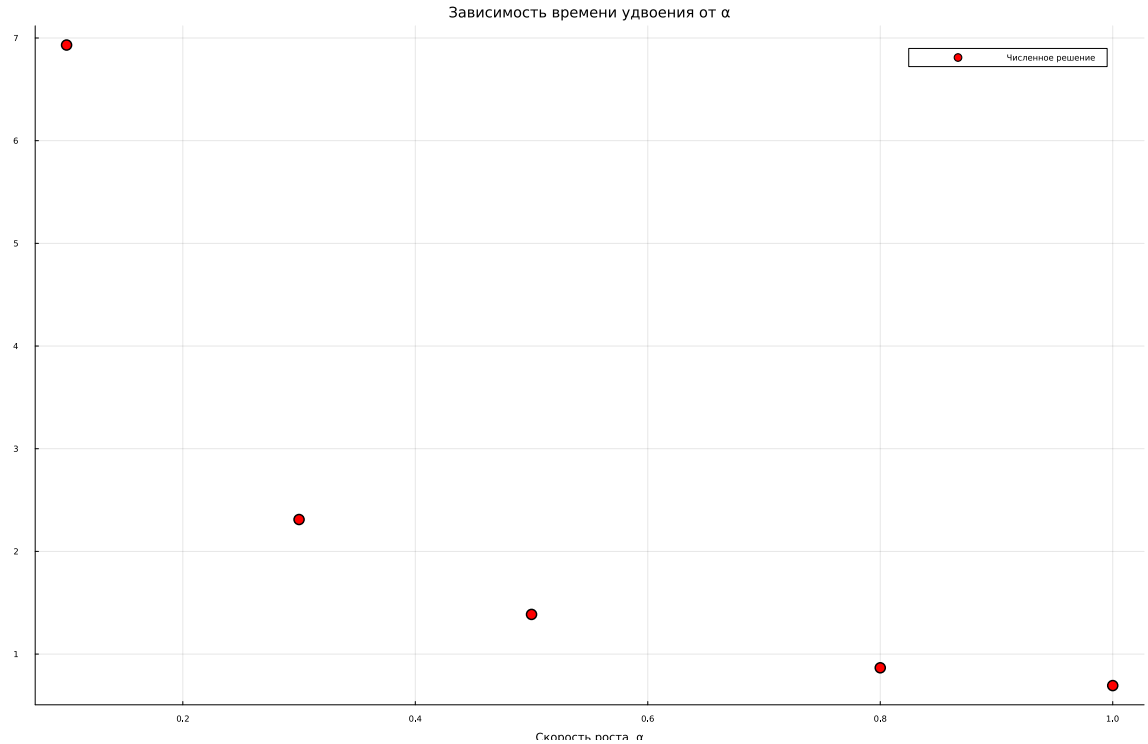
```
savefig(plotsdir(script_name, "parametric_scan_comparison.png"))
```

```
"C:\\Users\\arist\\Github\\study_2025-2026_mathmod\\labs\\lab01\\project\\plots\\para
```

График зависимости времени удвоения от α

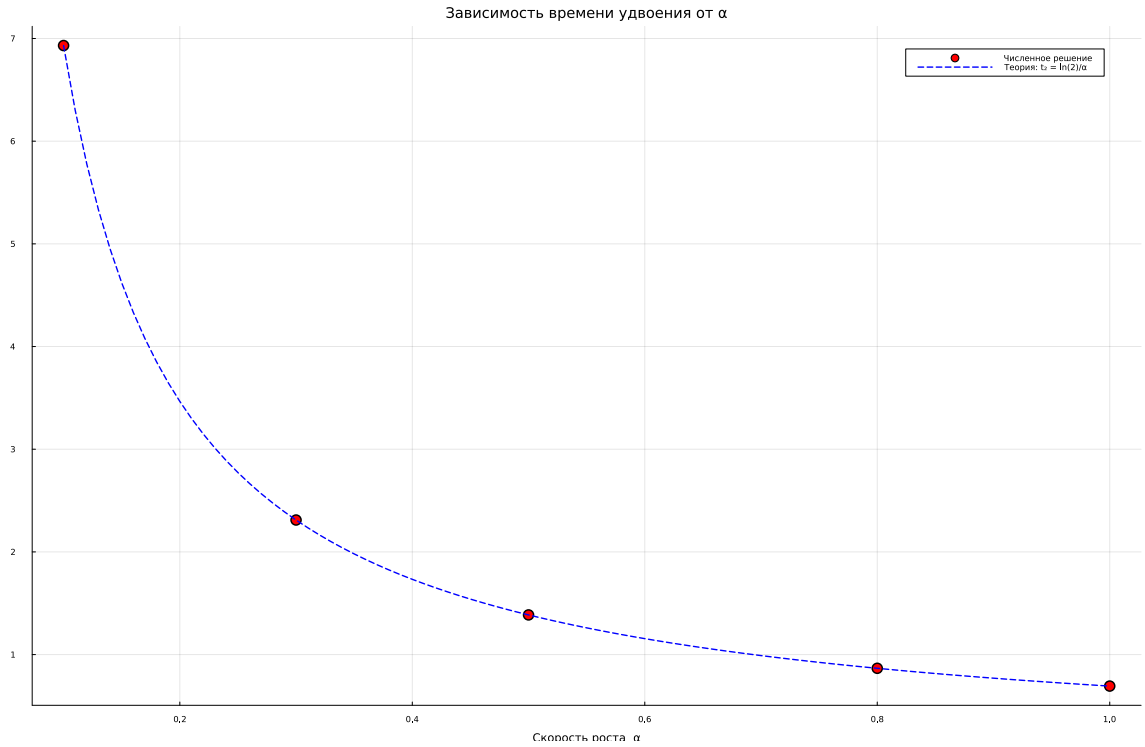
```
p3 = plot(results_df.□, results_df.doubling_time,
    seriestype=:scatter,
    label="Численное решение",
    xlabel="Скорость роста, □",
    ylabel="Время удвоения, t₂",
    title="Зависимость времени удвоения от □",
    markersize=8,
    markercolor=:red,
```

```
    legend=:topright
)
```



Теоретическая кривая: $t_2 = \ln(2)/\alpha$

```
α_range = 0.1:0.01:1.0

plot!(p3, α_range, log(2) ./ α_range,
    label="Теория: t₂ = ln(2)/α",
    lw=2,
    linestyle=:dash,
    linecolor=:blue
)
```



Сохраним график в папку plots

```
savefig(plotsdir(script_name, "doubling_time_vs_alpha.png"))
```

```
"C:\\Users\\arist\\Github\\study_2025-2026_mathmod\\labs\\lab01\\project\\plots\\doub
```

6.10 Бенчмаркинг с разными параметрами

ИЗМЕНЕНИЕ: Бенчмаркинг для разных значений α

```
println("\n" * "="^60)
println("Бенчмаркинг для разных значений □")
println("="^60)
benchmark_results = []
for □_value in param_grid[:□]
    bench_params = Dict(
```

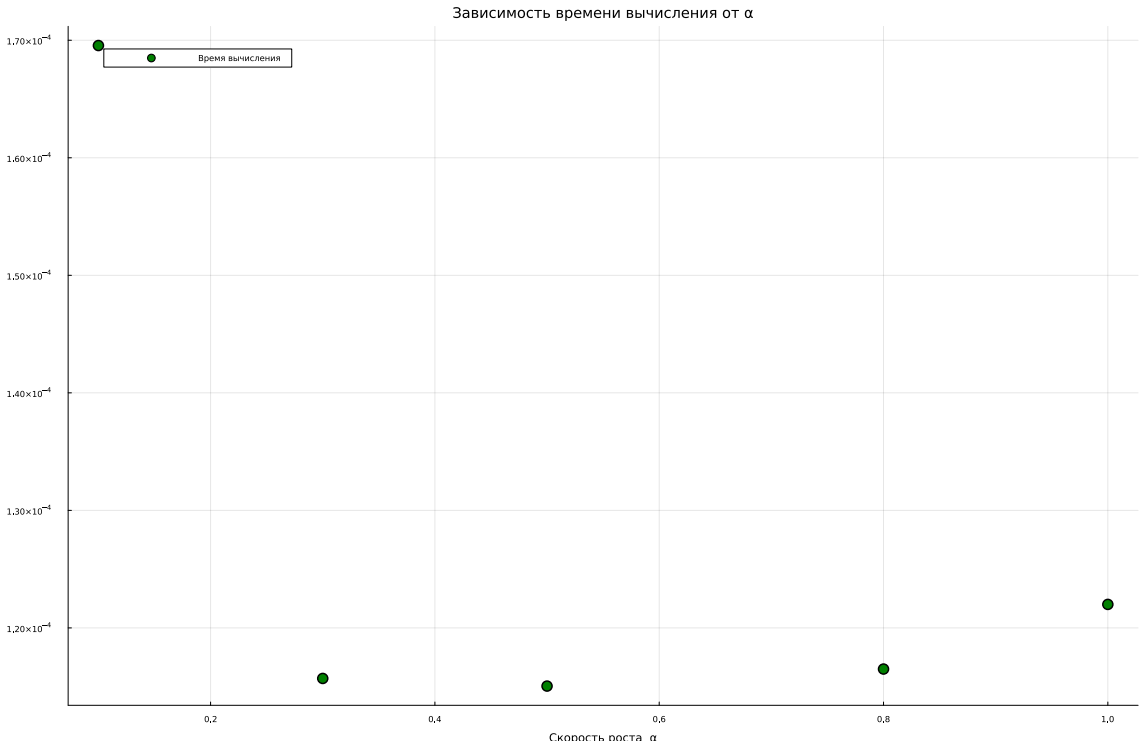
```
        :u0 => [1.0],
        :□ => □_value,
        :tspan => (0.0, 10.0),
        :solver => Tsit5(),
        :saveat => 0.1
) # Подготавливаем параметры для бенчмарка

    function benchmark_run() # Функция для бенчмарка
    prob = ODEProblem(exponential_growth!,
        bench_params[:u0],
        bench_params[:tspan],
        (□=bench_params[:□],))
    return solve(prob, bench_params[:solver];
        saveat=bench_params[:saveat])
    end
    println("\nБенчмарк для □ = $□_value:")
    b = @benchmark $benchmark_run() samples=100 evals=1 # Запуск бенчмарка
    push!(benchmark_results, (□=□_value, time=median(b).time/1e9)) # время в секундах
    println(" Среднее время: ", round(median(b).time/1e9; digits=4), " сек")
end
```

```
============================================================
Бенчмаркинг для разных значений □
============================================================

Бенчмарк для □ = 0.1:
 Среднее время: 0.0002 сек
```

```
Бенчмарк для □ = 0.3:
 Среднее время: 0.0001 сек

Бенчмарк для □ = 0.5:
 Среднее время: 0.0001 сек

Бенчмарк для □ = 0.8:
 Среднее время: 0.0001 сек

Бенчмарк для □ = 1.0:
 Среднее время: 0.0001 сек
```

График зависимости времени вычисления от α

```
bench_df = DataFrame(benchmark_results)
    p4 = plot(bench_df.□, bench_df.time,
    seriestype=:scatter,
    label="Время вычисления",
    xlabel="Скорость роста, □",
    ylabel="Время вычисления, сек",
    title="Зависимость времени вычисления от □",
    markersize=8,
    markercolor=:green,
    legend=:topleft
)
```



Сохраним график в папку plots

```
savefig(plotsdir(script_name, "computation_time_vs_alpha.png"))
```

```
"C:\\Users\\arist\\Github\\study_2025-2026_mathmod\\labs\\lab01\\project\\plots\\comp
```

6.11 Сохранение всех результатов

НОВАЯ СЕКЦИЯ: Сохранение сводных данных для последующего анализа

```
@save datadir(script_name, "all_results.jld2") base_params param_grid all_params resu
@save datadir(script_name, "all_plots.jld2") p1 p2 p3 p4
println("\n" * "="^60)
println("ЛАБОРАТОРНАЯ РАБОТА ЗАВЕРШЕНА")
println("="^60)
println("\nРезультаты сохранены в:")
```

```
println(" • data/$(script_name)/single/ - базовый эксперимент")
println(" • data/$(script_name)/parametric_scan/ - параметрическое сканирование")
println(" • data/$(script_name)/all_results.jld2 - сводные данные")
println(" • plots/$(script_name)/ - все графики")
println(" • data/$(script_name)/all_plots.jld2 - объекты графиков")
println("\nДля анализа результатов используйте:")
println(" using JLD2, DataFrames")

println(" @load \"data/$(script_name)/all_results.jld2\"")
println(" println(results_df)")
```

```
============================================================
ЛАБОРАТОРНАЯ РАБОТА ЗАВЕРШЕНА
============================================================

Результаты сохранены в:
 • data//single/ - базовый эксперимент
 • data//parametric_scan/ - параметрическое сканирование
 • data//all_results.jld2 - сводные данные
 • plots// - все графики
 • data//all_plots.jld2 - объекты графиков

Для анализа результатов используйте:
 using JLD2, DataFrames
 @load "data//all_results.jld2"
 println(results_df)
```

7 Выводы

1. Создано рабочее пространство курса на основе пакета DrWatson с установленными зависимостями для мат. моделировани.
2. Реализована модель экспоненциального роста в литературном стиле - код и документация совмещены в одном «.jl»-файле.
3. Из литетурного кода автоматически сгенерированы три формата: чистый скрипт, Jupyter Notebook, документа Quarty.
4. Реализация параметрических расчетов - проведено исследование модели при пяти разный значниях праметра α.
5. Итоговая документация объединяет код, результат вычисления и аналитические выводы в одном едином воспроизводимом формате.

Список литературы

1. A Multi-Language Computing Environment for Literate Programming and Reproducible Research / E. Schulte [et al.] // Journal of Statistical Software. — 2012. — Vol. 46, no. 3. — ISSN 1548-7660. — DOI: 10.18637/jss.v046.i03.

2. Knuth D. E. Literate Programming // The Computer Journal. — 1984. — Feb. — Vol. 27, no. 2. — P. 97–111. — ISSN 1460-2067. — DOI: 10.1093/comjnl/27.2.97.

3. The Story in the Notebook / M. B. Kery [et al.] // Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems. — ACM, 04/2018. — P. 1–11. — DOI: 10.1145/3173574.3173748.